

Day 47: Withdraw Method

Day 47 — Withdraw Method

1. Day Number

Day 47

2. Topic Name

Method with `if-elif-else`

Today you will learn how an object method can make a decision before changing an object's data.

3. Connection

Yesterday, you added a `deposit()` method to your `BankAccount` class.

The deposit method followed a simple rule:

```
If amount is positive
    add it to balance
```

Today, withdrawal needs a little more validation because two things must be checked:

```
Is the withdrawal amount valid?
    ↓
Is enough balance available?
    ↓
Only then reduce balance
```

So today you will add a `withdraw()` method.

4. Important Topics

You will practice:

- method
- `self`
- method parameter
- validation
- `if`
- `elif`
- `else`
- object attributes
- balance update
- preventing invalid operations

The main idea is:

```
Validate first
  ↓
Update data only when valid
```

5. Foundational Notes

What is a method?

A method is a function that belongs to a class.

For example:

```
class BankAccount:

    def withdraw(self, amount):
        ...
```

Here:

- withdraw is the method name
- self represents the current account object
- amount is the requested withdrawal amount

You might call it like:

```
account.withdraw(500)
```

Why does withdrawal need validation?

Imagine an account has:

```
Balance = ₹5000
```

Valid withdrawal

```
Withdraw = ₹1000
```

This can be allowed:

```
₹5000 - ₹1000 = ₹4000
```

Invalid withdrawal

```
Withdraw = ₹7000
```

The account does not have enough money.

Therefore:

Balance should remain ₹5000

Another invalid case

Withdraw = -₹500

A negative withdrawal does not make sense.

It must not change the balance.

Three possible situations

Your method can think about the amount in three stages:

```
Withdrawal request
    |
    v
Is amount <= 0?
  /      \
yes       no
|          |
invalid   v
          Is amount <= balance?
        /      \
       yes       no
       |          |
       withdraw insufficient balance
```

This is why if-elif-else is useful.

6. Easy Example

Consider a simple game player:

```
class Player:
    def __init__(self, energy):
        self.energy = energy
```

Suppose we want to use some energy.

The logic could be:

```
if requested energy is invalid:
    reject request

elif enough energy exists:
    reduce energy

else:
    say energy is not enough
```

For example:

```
Current energy = 100
Use energy = 30

New energy = 70
```

But:

```
Current energy = 100
Use energy = 150

Not enough energy
Energy remains 100
```

Your bank withdrawal method follows the same idea.

7. Problem Statement

Create or continue using your `BankAccount` class.

Add a method:

```
withdraw()
```

The method should accept a withdrawal amount.

Rules

Allow withdrawal only when:

```
amount > 0
```

and:

```
amount <= balance
```

If both conditions are valid:

```
balance = balance - amount
```

Otherwise, do not change the balance.

You should handle three cases:

```
amount <= 0
amount is valid and balance is enough
amount > balance
```

8. Concepts Used

Class

```
class BankAccount:
```

Represents the blueprint for bank-account objects.

Object

Example conceptually:

```
account = BankAccount(...)
```

The object stores its own balance.

Method

```
withdraw()
```

Provides withdrawal behavior to the account.

self

Used to access the current object's balance:

```
self.balance
```

Parameter

The method receives:

```
amount
```

Validation

Before changing the balance, check whether the requested amount is allowed.

if-elif-else

Used to handle multiple possible situations.

Attribute update

After successful withdrawal:

```
old balance → subtract amount → new balance
```

9. Thought Process

Suppose:

```
Account balance = ₹5000
```

Requested withdrawal = ₹2000

Think through the problem in this order.

Step 1: Check whether amount is positive

Ask:

Is ₹2000 greater than 0?

Yes.

Continue.

Step 2: Check available balance

Ask:

Is ₹2000 less than or equal to ₹5000?

Yes.

Continue.

Step 3: Update balance

Calculate:

5000 - 2000

New balance:

₹3000

Now consider:

Balance = ₹5000
Withdrawal = ₹7000

Step 1:

7000 > 0

Valid.

Step 2:

7000 <= 5000

False.

Therefore:

Do not change balance

For:

```
Withdrawal = 0
```

Immediately reject it because:

```
0 is not positive
```

10. Pseudocode

```
CLASS BankAccount

    constructor(account_holder, balance)
        store account_holder
        store balance

    METHOD withdraw(amount)

        IF amount is less than or equal to zero
            display invalid withdrawal message

        ELIF amount is less than or equal to balance
            subtract amount from balance
            display successful withdrawal message

        ELSE
            display insufficient balance message
```

Notice the important order:

```
Check invalid amount
    ↓
Check sufficient balance
    ↓
Otherwise insufficient balance
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Your class structure should conceptually look like:

```
BankAccount
|
|— account_holder
|— balance
|
|— display_account()
|— deposit()
|— withdraw()
```

The important idea is that the account manages its own balance.

Instead of changing balance directly from outside:

```
outside code → directly modify balance
```

prefer:

```
outside code
  |
  v
withdraw(amount)
  |
  v
validate request
  |
  v
update self.balance
```

This keeps the account behavior inside the BankAccount class.

12. Easy Edge Cases

Edge Case 1 — Withdrawal amount is 0

Example:

```
Balance: ₹5000
Withdraw: ₹0
```

Expected behavior:

```
Invalid withdrawal amount
Balance remains ₹5000
```

Because:

```
0 is not positive
```

Edge Case 2 — Negative withdrawal

Example:

```
Balance: ₹5000
Withdraw: ₹-500
```

Expected behavior:

```
Invalid withdrawal amount
Balance remains ₹5000
```


Edge Case 3 — Amount greater than balance

Example:

```
Balance: ₹5000  
Withdraw: ₹6000
```

Expected behavior:

```
Insufficient balance  
Balance remains ₹5000
```

Useful additional case — Withdraw complete balance

Suppose:

```
Balance: ₹5000  
Withdraw: ₹5000
```

This should normally be valid because:

```
withdraw amount <= balance
```

After withdrawal:

```
Balance: ₹0
```

So be careful not to accidentally use:

```
amount < balance
```

when the requirement allows withdrawing exactly the available balance.

13. Expected Input

For example, your object might initially contain:

```
Account holder: Radhe  
Balance: 5000
```

Then the withdrawal method might receive:

```
2000
```

Conceptually:

```
account.withdraw(2000)
```

Another test could be:

```
account.withdraw(7000)
```

Another:

```
account.withdraw(0)
```

14. Expected Output

For a valid withdrawal:

```
Account holder: Radhe
Starting balance: 5000

Withdrawal amount: 2000
Withdrawal successful

Remaining balance: 3000
```

For insufficient balance:

```
Starting balance: 5000

Withdrawal amount: 7000
Insufficient balance

Remaining balance: 5000
```

For invalid amount:

```
Starting balance: 5000

Withdrawal amount: 0
Invalid withdrawal amount

Remaining balance: 5000
```

The exact wording of your messages can be different. The important requirement is that **invalid withdrawals must not change the balance**.

15. Hint Only

Inside your `withdraw()` method, think about these three conditions:

```
if amount <= ...:
    # invalid amount

elif amount <= self.balance:
    # update balance

else:
    # insufficient balance
```

For the successful case, remember the balance needs to move in the opposite direction from yesterday's deposit:

Deposit:

$\text{balance} = \text{balance} + \text{amount}$

Withdraw:

$\text{balance} = \text{balance} - \text{amount}$

Try to complete the missing condition and balance update yourself without looking at a full solution.